GOOGLE CLOUD AI BUILDER CUP 2026 · BFSI

SENTINEL

V1 Build Checklist — Step by Step, Start to Submission

A complete, ordered, minute-level checklist for building the V1 prototype: card-not-present fraud investigation, one Cloud Run service, Gemini-powered Evidence Agent and Reasoning, Decision Robustness Scorer, deterministic Policy, and a real analyst UI.

Team: Vigilo | Product: Sentinel | Track: BFSI

Phase 0 — Before You Write Any Product Code

Do this first, in one sitting, before Week 1 begins

0.1 Accounts & access

- [ ] Create/confirm a Google Cloud project dedicated to Sentinel (not a shared personal project)
- [ ] Enable billing on the project and apply any AI Builder Cup GCP credits you've received
- [ ] Enable required APIs: Cloud Run, Vertex AI, Firestore, Secret Manager, Cloud Build, Artifact Registry
- [ ] Create a dedicated service account for the Cloud Run service with least-privilege roles (Vertex AI User, Firestore User — not Owner/Editor)
- [ ] Confirm Gemini model access in Vertex AI (test a single API call from the console before building anything)
- [ ] Set up Secret Manager entries for any API keys/config (even if empty placeholders for now)

0.2 Repository setup

- [ ] Create the repo with the agreed structure: `services/`, `shared/contracts`, `data/scenarios`, `data/seed`, `ui/`, `eval/`, `docs/`
- [ ] Add a `README.md` stating: product name, one-paragraph pitch, architecture diagram, how to run locally
- [ ] Set up a Python virtual environment (or your chosen stack) with a pinned `requirements.txt` / `pyproject.toml`
- [ ] Add a `.env.example` documenting every required environment variable (Gemini model names, GCP project ID, Firestore collection names)
- [ ] Set up basic CI (even just a lint + test run on push) so broken code doesn't silently sit for days
- [ ] Add all frozen contract documents (Transaction, EvidenceBundle, CaseFindings, PolicyDecision, AuditEvent) into `shared/contracts/` as the literal source of truth — code should import from here, not redefine shapes ad hoc

0.3 Domain scope lock-in

> This must be settled before any schema code is written — revisiting it mid-build is expensive.

- [ ] Confirm scenario: Card-Not-Present (CNP) purchase, outgoing, pre-authorization stage
- [ ] Write down, in the README, the one sentence: "Sentinel V1 investigates outgoing CNP card purchases at the pre-authorization stage."
- [ ] Confirm which fields are mandatory on `Transaction`: `payment_rail`, `direction`, `authorization_stage`, `card_present`
- [ ] Confirm the six entities in scope: Customer, Account, Card/Payment Instrument, Merchant, Device, IP/Location

Phase 1 — Data & Detection

Week 1

1.1 Real labeled dataset

- [ ] Download a real public labeled fraud dataset (e.g. Kaggle Credit Card Fraud Detection, or IEEE-CIS Fraud Detection, or PaySim)
- [ ] Document the dataset source, size, fraud/legit ratio, and license in `data/README.md`
- [ ] Split into train / held-out evaluation sets (e.g. 80/20), and never let the held-out split touch training or fixture generation
- [ ] Clean and normalize columns to roughly map onto your `Transaction` schema fields (amount, timestamp, merchant category, etc.)

1.2 Detection model

- [ ] Choose a simple, defensible model type (gradient-boosted trees or logistic regression — do not over-engineer this)
- [ ] Train the model on the training split; log `model_name` and `model_version` as artifacts, not just in memory
- [ ] Evaluate on the held-out split: compute precision, recall, ROC-AUC, PR-AUC, false-negative rate
- [ ] Save these numbers to a file (`eval/detection_baseline.json`) — this is what goes into your submission later
- [ ] Wrap the trained model behind a simple `predict(transaction) -> risk_score` function
- [ ] Define the `risk_band` mapping (LOW/MEDIUM/HIGH) from `risk_score` thresholds — pick thresholds from the score distribution on held-out data, not guessed numbers

1.3 Deterministic anomaly tagging

- [ ] Implement anomaly tagging as plain code (not a model): `NEW_DEVICE`, `UNUSUAL_LOCATION`, `UNUSUAL_AMOUNT`, `MERCHANT_RISK`, `LINKED_ENTITY_ANOMALY` (drop tags irrelevant to CNP scope: `IMPOSSIBLE_TRAVEL`, `KYC_ANOMALY` optional for V1)
- [ ] Write one pure function per tag: given transaction + fixture context, return true/false
- [ ] Unit test each tag function independently with at least one true and one false example
- [ ] Assemble `DetectionOutput`: `case_id`, `transaction_id`, `risk_score`, `risk_band`, `anomaly_tags[]`, `detection_reasons[]`, `model_version`, `detected_at`

1.4 Fixture / seed data — this is the most important sub-task in Phase 1

> The branching-proof scenario depends entirely on this fixture data being seeded correctly. Do not rush this.

- [ ] Create seed fixtures for: 3–5 customers, their accounts, cards, devices, and behavioral baselines
- [ ] Create seed fixtures for: 3–5 merchants, including one with a rising fraud-report trend
- [ ] Build the **clean/normal transaction scenario**: no anomalies, should AUTO_RESOLVE
- [ ] Build the **signature fraud scenario**: new device, unusual location, risky merchant — multi-signal, should ESCALATE
- [ ] Build the **branching-proof scenario** specifically:
  - [ ] Transaction with only `NEW_DEVICE` tag firing at detection time
  - [ ] Seed `get_device_history(device_id)` fixture to return: this device was also seen on a *different* account 2 hours earlier
  - [ ] Seed `get_linked_entities` fixture so that following this thread reveals a recent anomaly on the linked account
  - [ ] Confirm nothing in the minimum-evidence floor for `NEW_DEVICE` alone would force this tool call — the agent must choose it
- [ ] Build a **PARTIAL-evidence scenario**: one tool deliberately configured to fail/timeout, to prove the fail-safe escalation path
- [ ] Build a **merchant-risk-trend scenario**: rising fraud-report count at a new-to-customer merchant, used for the Robustness Scorer demo variant
- [ ] Store all fixtures as versioned JSON/YAML files in `data/seed/`, loaded deterministically by a fixed seed value

Phase 2 — Evidence Agent

Week 2 — build this before anything downstream

> This is your highest-risk, highest-value component. If the branching behavior doesn't work reliably, your core "genuine agent" claim collapses. Do not move to Phase 3 until this phase's exit checklist is fully checked.

2.1 Tool layer (read-only, mocked)

- [ ] Implement `get_device_history(device_id, customer_id?)` against fixture data
- [ ] Implement `get_ip_geovelocity(customer_id, ip_address, transaction_time)` against fixture data
- [ ] Implement `get_behavioral_baseline(customer_id, transaction)` against fixture data
- [ ] Implement `get_linked_entities(entity_id, relation_scope)` against fixture data
- [ ] Implement `get_merchant_risk(merchant_id, customer_id)` returning the full v1.1 shape: `category`, `onboarding_date`, `chargeback_rate`, `recent_fraud_report_count`, `fraud_report_trend`, `customer_prior_transactions_with_merchant`, `risk_band`
- [ ] Implement `get_mock_kyc_status(customer_id)` (optional for CNP scope, keep if cheap)
- [ ] Wrap every tool call with: input schema validation, per-tool timeout, structured error on failure (never a silent exception)
- [ ] Normalize every tool output into a `Fact` object with `fact_id`, `source_tool`, `observed_at`, `retrieved_at`

2.2 Minimum-evidence floor

- [ ] Implement the floor table in code: `NEW_DEVICE → get_device_history`, `UNUSUAL_LOCATION → get_ip_geovelocity`, `MERCHANT_RISK → get_merchant_risk`, `UNUSUAL_AMOUNT → get_behavioral_baseline`, `LINKED_ENTITY_ANOMALY → get_linked_entities`
- [ ] Implement floor union logic: for multiple anomaly tags, required tools = union of each tag's requirement
- [ ] Write a function `is_floor_satisfied(tools_called, required_tools) -> bool` and unit test it directly, independent of the agent loop

2.3 Bounded Gemini function-calling loop

- [ ] Define the tool schemas Gemini will see (name, description, input parameters) for all six tools
- [ ] Configure Gemini Flash for this component specifically (not Pro — cost/latency reason, state this explicitly in code comments)
- [ ] Implement the loop: read case + evidence so far → let Gemini choose next tool or declare sufficiency → execute → normalize into facts → repeat
- [ ] Enforce hard cap: max 6 tool calls
- [ ] Enforce a total investigation deadline (wall-clock timeout across the whole loop)
- [ ] Enforce the explicit tool allow-list — reject any tool name Gemini returns that isn't in the six above
- [ ] Implement `stop_reason` logic: `SUFFICIENT_EVIDENCE`, `MAX_CALLS`, `TIMEOUT`, `TOOL_FAILURE`, `FLOOR_SATISFIED`
- [ ] Confirm the agent cannot mark `completeness = COMPLETE` until the minimum-evidence floor is satisfied, even if Gemini "thinks" it's done earlier

2.4 Prove the branching behavior — exit criteria for this phase

- [ ] Run the branching-proof scenario end to end
- [ ] Confirm the agent calls `get_device_history` first (required by the floor)
- [ ] Confirm the agent then *chooses* to call `get_linked_entities` — a tool the original `NEW_DEVICE` floor did not require — because the device-history result revealed a relationship worth following
- [ ] Re-run this exact scenario at least 10 times; confirm the branching happens reliably, not occasionally
- [ ] Log and save the tool-call sequence from these runs as evidence for your submission ("Evidence Agent branching success rate: X/10")
- [ ] Write a short internal note describing exactly what would make this a fake/fixed-mapping agent instead of a real one — confirm your implementation does not match that failure pattern

Phase 3 — Reasoning & Decision Robustness Scorer

Week 3

3.1 Gemini Pro reasoning call

- [ ] Configure Gemini Pro for this component specifically (higher reasoning load, lower call volume — state this explicitly)
- [ ] Build the `ReasoningInput` assembly: transaction, detection output, full evidence bundle, model/prompt version tags
- [ ] Write the prompt template so evidence is passed as clearly delimited structured data, never concatenated as free text into instruction position
- [ ] Request structured output matching `CaseFindings`: `risk_assessment`, `summary`, `supporting_factors[]` (with `materiality`), `counter_factors[]`, `evidence_gaps[]`, `evidence_citations[]`, `recommended_disposition`
- [ ] Enforce schema validation on the response before accepting it

3.2 Citation validation

- [ ] For every claim/factor in the output, verify each referenced `fact_id` actually exists in the evidence bundle passed in
- [ ] Reject and reprompt (schema-only, stricter instruction) if any citation is invalid or missing
- [ ] On second failure, mark `reasoning.status = FAILED` and let this flow into Policy as a forced escalation
- [ ] Write a small test suite: valid output passes, output with a fabricated fact_id is rejected

3.3 Decision Robustness Scorer

- [ ] Implement the fast-path scorer using `materiality`: for each HIGH-materiality supporting factor, estimate whether removing it would flip `risk_assessment` below the Policy escalation threshold
- [ ] Classify each case as `ROBUST` (no single factor removal flips the outcome) or `FRAGILE` (one factor's removal does), and store `fragile_factor` if applicable
- [ ] Implement the live counterfactual path separately: given a specific fact to remove, rebuild the evidence bundle without it and re-call Reasoning for real — this is the one used in the live demo moment, not the fast-path estimate
- [ ] Test both paths against the branching-proof scenario and the merchant-risk-trend scenario specifically — confirm you get a genuine FRAGILE result on at least one seeded variant, not only ROBUST results everywhere

Phase 4 — Policy, Firestore & Pipeline Wiring

Week 4

4.1 Deterministic Policy engine

- [ ] Implement `PolicyRule` as versioned data (not embedded if/else logic scattered in code) — a list of rules with `rule_id`, `version`, `priority`, `condition`, `outcome`, `effective_from`, `enabled`
- [ ] Implement the rule evaluator as a pure function: inputs → exactly one of `AUTO_RESOLVE` / `ESCALATE`, never anything else
- [ ] Encode the five rules: partial evidence → escalate; reasoning failed → escalate; high risk + high value + complete evidence → escalate; risk_score medium+ AND risk_assessment HIGH + complete evidence → escalate; low risk + complete + no hard tag + risk_assessment not HIGH → auto-resolve; otherwise → escalate
- [ ] Confirm in code review that no path can produce `BLOCK_HOLD` from this component — search the codebase for this literal string outside the human-decision contract
- [ ] Write unit tests for every rule, including edge cases at each threshold boundary
- [ ] Record `rule_id`, `rule_version`, `reasons[]`, `inputs_snapshot` on every `PolicyDecision`

4.2 Firestore case store

- [ ] Design the single `cases/{case_id}` document shape: transaction, detection, evidence_bundle, findings, robustness, policy_decision, state, assigned_analyst_id
- [ ] Add an `audit_events` subcollection per case; write one entry on every state transition
- [ ] Implement the case state machine transitions server-side only — confirm no client/UI code can set state directly
- [ ] Implement the uniqueness rule: reject creating a second active case for a `transaction_id` that already has one open
- [ ] Set up Firestore security rules: analyst UI can read cases and write only via the defined backend action endpoints, never direct field writes

4.3 Wire the full pipeline end to end

- [ ] Build the single entry point: `POST /transactions` → Detection → (if flagged) Evidence Agent → Reasoning → Robustness Scorer → Policy → Firestore write
- [ ] Confirm clean/normal transactions skip investigation entirely and resolve immediately
- [ ] Run all seeded scenarios from Phase 1.4 through the full pipeline; confirm each produces the expected outcome
- [ ] Add structured logging with `case_id` and `transaction_id` on every log line across every stage
- [ ] Deploy this version to Cloud Run for the first time — don't wait until Week 5 to test real deployment

Phase 5 — Analyst UI & Evaluation Numbers

Week 5

5.1 Case API (backend for the UI)

- [ ] Implement `GET /cases` (queue view — escalated cases only, with risk band and robustness flag)
- [ ] Implement `GET /cases/{case_id}` (full case detail)
- [ ] Implement `POST /cases/{case_id}/decision` accepting `analyst_id`, `action` (CLEAR/BLOCK_HOLD/ESCALATE_SENIOR), `reason`, `evidence_acknowledged`
- [ ] Enforce: case must be in `HUMAN_REVIEW` state; reason required for BLOCK_HOLD and ESCALATE_SENIOR; action is idempotent; writes an audit event

5.2 Queue view (screen 1)

- [ ] List of escalated cases: transaction amount, merchant, risk band, robustness flag (ROBUST/FRAGILE) visible at a glance
- [ ] Sort/filter by risk band or robustness as a minimum interaction
- [ ] Visually distinct treatment for FRAGILE cases — this should look different from ROBUST at a glance, not require reading text

5.3 Case detail view (screen 2)

- [ ] Transaction summary block: amount, merchant, device, location, card info
- [ ] Evidence cards, one per fact, each tagged with its source tool
- [ ] Gemini findings: summary, supporting/counter factors, with citations visibly linked to the evidence cards above
- [ ] Decision Robustness Score displayed prominently, with the fragile factor named if applicable
- [ ] Policy section: which rule fired, rule version, outcome
- [ ] Three action buttons: CLEAR / BLOCK_HOLD / ESCALATE_SENIOR, each requiring a reason before submission
- [ ] An explicit "evidence acknowledged" checkbox/gate before actions become available

5.4 Design pass — do not skip this

- [ ] Apply a real visual design system (consistent spacing, type scale, a real color palette tied to your Sentinel brand) — not default component styling
- [ ] Test the UI on the actual screen size/resolution you'll demo on
- [ ] Get one outside pair of eyes to use it without explanation and see if they understand the case at a glance

5.5 Evaluation numbers — capture and save these

- [ ] Detection: precision, recall, ROC-AUC, PR-AUC on held-out data (from Phase 1)
- [ ] Evidence Agent: branching-scenario success rate across repeated runs (from Phase 2.4)
- [ ] Reasoning: schema-validity rate and citation-validity rate across your full scenario set
- [ ] End-to-end: for each of your ~10–20 seeded scenarios, does the Policy outcome match what a reasonable human analyst would call correct? Build this as a small table
- [ ] Save every number into a single `eval/results_summary.md` file — this becomes a slide and a submission section, not something you reconstruct at the last minute

Phase 6 — Demo Hardening & Submission

Final days before deadline

6.1 Demo script rehearsal

- [ ] Script the 2.5-minute demo exactly: transaction submitted → Evidence Agent visible branching → Reasoning findings → Robustness Score (FRAGILE) → live counterfactual re-run → score/outcome flips → Policy rule shown → escalation → analyst action
- [ ] Rehearse this full sequence at least 10 times end to end, on the actual machine/network you'll demo from
- [ ] Time every rehearsal; know exactly where the slow points are

6.2 Fallback path

- [ ] Build a strict timeout around the live counterfactual re-run in the demo
- [ ] Pre-cache the known-good result for your exact seeded scenario as a fallback
- [ ] Test the fallback path deliberately (simulate a slow/failed Gemini call) and confirm it triggers cleanly
- [ ] Decide, and rehearse, exactly what you'll say if the fallback triggers live — state it plainly as a fallback, never pretend it's live

6.3 Reliability pass

- [ ] Test: Gemini call fails/times out → confirm bounded retry then escalation, not a crash
- [ ] Test: a tool call fails → confirm partial evidence, never silent auto-resolve
- [ ] Test: malformed Gemini output (simulate) → confirm one reprompt, then escalation
- [ ] Test: duplicate transaction submission → confirm no duplicate active case is created
- [ ] Run the full clean-state pipeline at least 5 times back to back with no manual intervention

6.4 Submission package

- [ ] Deploy the final version to Cloud Run and confirm the public/demo URL works from a fresh browser session
- [ ] Record a backup demo video in case of live connectivity issues on presentation day
- [ ] Write the submission narrative leading with: the branching-agent proof, the Decision Robustness Score, then the architecture — in that order, not diagram-first
- [ ] Include the real evaluation numbers table from Phase 5.5
- [ ] Explicitly name, in one section, what's mocked/synthetic for the hackathon vs. what a production version would use (merchant data, KYC, device history) — state this proactively, don't let a judge find it
- [ ] Prepare one-line pitches for: Best use of Google Cloud AI tools, Most Impactful Solution, Best UI/UX — tie each explicitly to a specific thing you built
- [ ] Proofread every field of the submission form itself before final submit
- [ ] Submit with time to spare — do not submit in the final hour

6.5 Final gate — do not submit until every item below is checked

- [ ] Detection trained and evaluated on real labeled data, numbers captured
- [ ] Branching-proof scenario passes reliably across repeated runs
- [ ] Evidence Agent enforces the minimum-evidence floor in code, not prompt
- [ ] Reasoning output is schema- and citation-validated with a working retry-then-escalate path
- [ ] Decision Robustness Score computes correctly and the live counterfactual works under demo conditions
- [ ] Policy never produces anything but AUTO_RESOLVE/ESCALATE — verified by code search, not assumption
- [ ] Analyst UI has both real screens, three human actions, reason required
- [ ] Full pipeline runs end-to-end from a clean state, repeatably, on command
- [ ] A tested pre-cached fallback exists for the live demo moment
- [ ] Real evaluation numbers are in the submission, not just claimed